

Theoretical Background

Geometry

The physical domain consists of two infinite parallel plates separated by a constant distance H . The coordinate system is defined such that the x -axis is aligned with the streamwise flow direction, and the y -axis is normal to the plates.

- **Domain:** $0 \leq y \leq H$ and $-\infty < x < \infty$
- **Top Plate** ($y = H$): Stationary (velocity $U = 0$)
- **Bottom Plate** ($y = 0$): Moving with a constant velocity U_w

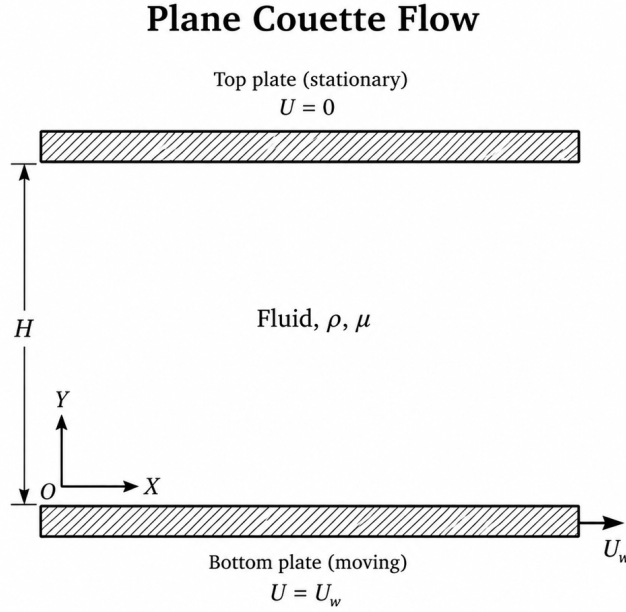


Figure 1: Channel Geometry

As illustrated in Figure 1, the fluid occupies the region between the two parallel plates, with the origin of the coordinate system located on the bottom plate.

Governing Equations

To derive the mathematical model for plane Couette flow, the following assumptions are adopted:

1. The fluid is incompressible, and its density ρ is constant.
2. The thermophysical properties of the fluid are constant, including the dynamic viscosity μ .
3. The flow is one-dimensional; therefore, the velocity has only an x -component.
4. The plates are infinitely long in the flow direction.

5. No external pressure gradient is imposed in the streamwise direction, such that $\partial p / \partial x = 0$.
6. Gravitational effects and other body forces are neglected.

Continuity Equation

The general form of the continuity equation is:

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{V}) = 0 \quad (1)$$

For a two-dimensional Cartesian coordinate system, Equation (1) is expressed as:

$$\frac{\partial \rho}{\partial t} + \frac{\partial(\rho u)}{\partial x} + \frac{\partial(\rho v)}{\partial y} = 0 \quad (2)$$

Given that the density is constant, $\partial \rho / \partial t = 0$, Equation (2) reduces to:

$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0 \quad (3)$$

For the one-dimensional plane Couette flow considered here, the velocity field has only a streamwise component:

$$\mathbf{V} = [u(y, t), 0]$$

Therefore, the velocity component normal to the plates is identically zero:

$$v = 0$$

Thus, Equation (3) simplifies to:

$$\frac{\partial u}{\partial x} = 0 \quad (4)$$

It follows from Equation (4) that the streamwise velocity is independent of x and can be written as:

$$u = u(y, t)$$

Momentum Equation

For an incompressible Newtonian fluid with body-force effects neglected, the Navier–Stokes momentum equation is:

$$\rho \left(\frac{\partial \mathbf{V}}{\partial t} + \mathbf{V} \cdot \nabla \mathbf{V} \right) = -\nabla p + \mu \nabla^2 \mathbf{V} \quad (5)$$

Given the velocity field $\mathbf{V} = [u(y, t), 0]$, the x -momentum component becomes:

$$\rho \left(\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} + v \frac{\partial u}{\partial y} \right) = -\frac{\partial p}{\partial x} + \mu \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right) \quad (6)$$

Substituting the kinematic conditions:

$$\frac{\partial u}{\partial x} = 0, \quad v = 0, \quad \frac{\partial^2 u}{\partial x^2} = 0$$

together with the zero streamwise pressure gradient:

$$\frac{\partial p}{\partial x} = 0$$

into Equation (6) yields:

$$\rho \frac{\partial u}{\partial t} = \mu \frac{\partial^2 u}{\partial y^2} \quad (7)$$

Dividing Equation (7) by the density gives the one-dimensional transient diffusion equation:

$$\frac{\partial u}{\partial t} = \nu \frac{\partial^2 u}{\partial y^2} \quad (8)$$

where:

$$\nu = \frac{\mu}{\rho}$$

is the kinematic viscosity.

Additionally, the y -momentum equation reduces to:

$$\frac{\partial p}{\partial y} = 0$$

which confirms that the pressure is uniform across the channel height.

Analytical Solution

(This section is reserved for the analytical solution comparison.)

Initial and Boundary Conditions

The initial and boundary conditions for the transient plane Couette flow problem are defined as follows.

Initial Condition

At the initial time $t = 0$, the fluid is at rest throughout the channel interior, while the bottom plate is already moving with the constant velocity U_w :

$$u(0, 0) = U_w$$

$$u(y, 0) = 0 \quad \text{for } 0 < y \leq H$$

Boundary Conditions

For $t \geq 0$, the velocity field satisfies the following Dirichlet boundary conditions:

$$u(0, t) = U_w$$

$$u(H, t) = 0$$

The first condition represents the no-slip condition at the moving bottom plate, while the second condition represents the no-slip condition at the stationary top plate. Together, these conditions describe the impulsive start of the bottom plate and the subsequent transient diffusion of momentum across the channel.

Domain Discretization

To solve the governing diffusion equation numerically, the continuous spatial domain $0 \leq y \leq H$ is discretized into a finite number of grid points (nodes). Let N represent the total number of nodes along the y -direction. The uniform distance between consecutive nodes, referred to as the grid spacing Δy , is defined as:

$$\Delta y = \frac{H}{N - 1}$$

The discrete spatial coordinates of these grid points are given by:

$$y_j = j\Delta y, \quad j = 0, 1, 2, \dots, N-1$$

In alignment with C++ indexing standards, the index j starts from 0 (bottom wall) and ends at $N-1$ (top wall). The spatial discretization of the domain is illustrated in Figure 2.

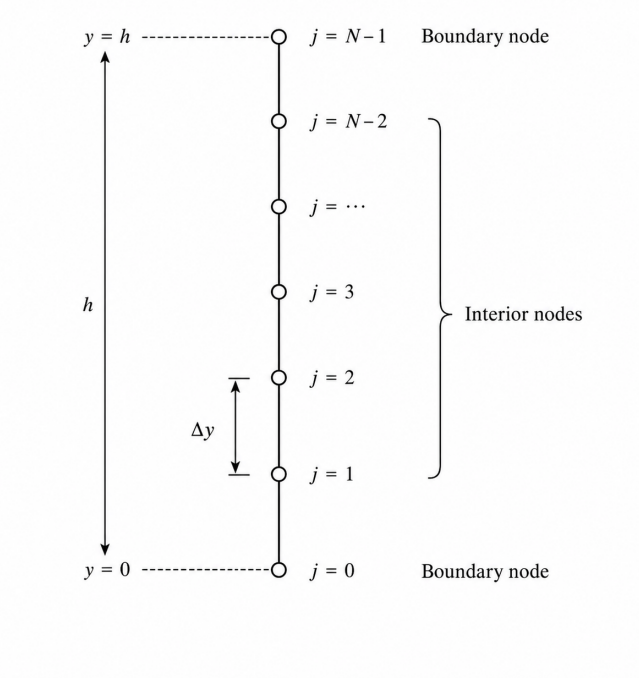


Figure 2: Computational Grid

Similarly, the temporal domain is discretized into uniform intervals of Δt , where n denotes the time level. The continuous velocity $u(y, t)$ at a specific grid location y_j and time $t^n = n\Delta t$ is represented using the following discrete notation:

$$u(y_j, t^n) \approx u_j^n$$

This discrete representation serves as the foundation for approximating the partial derivatives in the governing equation using Finite Difference Methods (FDM).

Numerical Schemes

In this section, the transition from the continuous partial differential equation to discrete algebraic forms suitable for C++ implementation is presented.

To solve the one-dimensional transient diffusion equation, previously introduced in Theoretical Background as Equation (9):

$$\frac{\partial u}{\partial t} = \nu \frac{\partial^2 u}{\partial y^2} \quad (9)$$

the continuous temporal and spatial derivatives must be approximated using finite difference expressions. Implicit numerical schemes are therefore introduced to obtain discrete formulations of the governing equation.

Implicit Numerical Schemes

1. **Laasonen Scheme**
2. **CrankNicolson Scheme**

Laasonen Scheme

The Implicit Euler scheme, commonly known as the Laasonen scheme, is an implicit finite difference method for solving the one-dimensional transient diffusion equation.

Unlike the FTCS and Richardson schemes, which evaluate the spatial derivative using only previously known values, the Laasonen scheme evaluates the spatial derivative at the new time level $n+1$. Consequently, the unknown values at the new time level appear simultaneously in the discretized equation, and a system of linear algebraic equations must be solved at every time step.

The governing one-dimensional diffusion equation, evaluated at the grid point (y_j, t^{n+1}) , gives:

$$\left(\frac{\partial u}{\partial t}\right)_j^{n+1} = \nu \left(\frac{\partial^2 u}{\partial y^2}\right)_j^{n+1} \quad (10)$$

The Laasonen scheme uses:

- a first-order backward difference in time;
- a second-order central difference in space;
- an implicit evaluation of the diffusion term at time level $n+1$;
- a tridiagonal linear system at each time step.

A. Discretization of the First-Order Temporal Derivative

To approximate the first-order temporal derivative at time level $n+1$, the value u_j^n is expanded about the point (y_j, t^{n+1}) :

$$u_j^n = u_j^{n+1} - \left(\frac{\partial u}{\partial t}\right)_j^{n+1} \Delta t + \frac{1}{2!} \left(\frac{\partial^2 u}{\partial t^2}\right)_j^{n+1} (\Delta t)^2 - \frac{1}{3!} \left(\frac{\partial^3 u}{\partial t^3}\right)_j^{n+1} (\Delta t)^3 + \mathcal{O}((\Delta t)^4) \quad (11)$$

Rearranging Equation (11) to isolate the first-order temporal derivative gives:

$$\left(\frac{\partial u}{\partial t}\right)_j^{n+1} = \frac{u_j^{n+1} - u_j^n}{\Delta t} + \mathcal{O}(\Delta t) \quad (12)$$

Therefore, the backward difference approximation is first-order accurate in time.

B. Discretization of the Second-Order Spatial Derivative

The second-order spatial derivative is evaluated at time level $n + 1$. Expanding the neighboring values u_{j+1}^{n+1} and u_{j-1}^{n+1} about (y_j, t^{n+1}) gives:

$$u_{j+1}^{n+1} = u_j^{n+1} + \left(\frac{\partial u}{\partial y}\right)_j^{n+1} \Delta y + \frac{1}{2!} \left(\frac{\partial^2 u}{\partial y^2}\right)_j^{n+1} (\Delta y)^2 + \frac{1}{3!} \left(\frac{\partial^3 u}{\partial y^3}\right)_j^{n+1} (\Delta y)^3 + \frac{1}{4!} \left(\frac{\partial^4 u}{\partial y^4}\right)_j^{n+1} (\Delta y)^4 + \dots \quad (13)$$

$$u_{j-1}^{n+1} = u_j^{n+1} - \left(\frac{\partial u}{\partial y}\right)_j^{n+1} \Delta y + \frac{1}{2!} \left(\frac{\partial^2 u}{\partial y^2}\right)_j^{n+1} (\Delta y)^2 - \frac{1}{3!} \left(\frac{\partial^3 u}{\partial y^3}\right)_j^{n+1} (\Delta y)^3 + \frac{1}{4!} \left(\frac{\partial^4 u}{\partial y^4}\right)_j^{n+1} (\Delta y)^4 + \dots \quad (14)$$

Adding Equations (13) and (14) eliminates the odd-order spatial derivative terms:

$$u_{j+1}^{n+1} + u_{j-1}^{n+1} = 2u_j^{n+1} + \left(\frac{\partial^2 u}{\partial y^2}\right)_j^{n+1} (\Delta y)^2 + \frac{1}{12} \left(\frac{\partial^4 u}{\partial y^4}\right)_j^{n+1} (\Delta y)^4 + \dots \quad (15)$$

Rearranging Equation (15) gives the second-order central difference approximation:

$$\left(\frac{\partial^2 u}{\partial y^2}\right)_j^{n+1} = \frac{u_{j+1}^{n+1} - 2u_j^{n+1} + u_{j-1}^{n+1}}{(\Delta y)^2} + \mathcal{O}((\Delta y)^2) \quad (16)$$

Thus, the spatial discretization is second-order accurate.

C. Discrete Algebraic Equation

Substituting the temporal approximation from Equation (12) and the spatial approximation from Equation (16) into the governing equation in Equation (10) gives:

$$\frac{u_j^{n+1} - u_j^n}{\Delta t} = \nu \left(\frac{u_{j+1}^{n+1} - 2u_j^{n+1} + u_{j-1}^{n+1}}{(\Delta y)^2} \right) \quad (17)$$

Define the dimensionless diffusion number as:

$$d = \frac{\nu \Delta t}{(\Delta y)^2} \quad (18)$$

Multiplying Equation (17) by Δt and using Equation (18) yields:

$$u_j^{n+1} - u_j^n = d \left(u_{j+1}^{n+1} - 2u_j^{n+1} + u_{j-1}^{n+1} \right) \quad (19)$$

Rearranging Equation (19) so that all unknown values at time level $n + 1$ are placed on the left-hand side gives:

$$-d u_{j-1}^{n+1} + (1 + 2d) u_j^{n+1} - d u_{j+1}^{n+1} = u_j^n \quad (20)$$

Equation (20) is the discrete algebraic form of the Laasonen scheme. It must be solved simultaneously for all spatial nodes at every time step. At every time step, the complete program must:

- Initialize the solution vector u_j^n from the initial condition.
- Apply the boundary conditions on both sides.
- Assemble and solve the tridiagonal system.
- Update the solution to the next time level.

D. Accuracy of the Laasonen Scheme

The temporal truncation error is obtained from Equation (12):

$$\text{Temporal truncation error} = \mathcal{O}(\Delta t)$$

The spatial truncation error is obtained from Equation (16):

$$\text{Spatial truncation error} = \mathcal{O}((\Delta y)^2)$$

Therefore, the Laasonen scheme is:

- first-order accurate in time;
- second-order accurate in space.

The combined local truncation error can be written as:

$$\mathcal{T}_{\text{Laasonen}} = \mathcal{O}(\Delta t) + \mathcal{O}((\Delta y)^2) \quad (21)$$

The spatial accuracy of Laasonen is the same as that of FTCS, while its temporal accuracy is the same order as FTCS. The main difference between the two methods is their stability behavior and algebraic structure.

E. Comparison with the Explicit Schemes

The main characteristics of the four schemes considered so far are summarized below:

Scheme	Time discretization	Spatial discretization	Algebraic system	Temporal accuracy	Stability
FTCS	Forward explicit	Central	Not required	First-order	Conditionally stable, $d \leq \frac{1}{2}$
Richardson	Central explicit	Central	Not required	Second-order	Unconditionally unstable
DuFort-Frankel	Central explicit	Modified central	Not required	Nominally second-order	Unconditionally stable
Laasonen	Backward implicit	Central	Tridiagonal system	First-order	Unconditionally stable

The Laasonen scheme has the same formal temporal accuracy as the FTCS scheme, but it does not impose the FTCS stability restriction. Its main computational requirement is the solution of a tridiagonal linear system at every time step.

In contrast:

- FTCS is simple to implement because it requires only a direct explicit update.
- Richardson has second-order formal accuracy in both time and space, but it is unusable because of the continuous instability.
- DuFort-Frankel is explicit and unconditionally stable, but it uses three time levels and requires a starting procedure.
- Laasonen is implicit and unconditionally stable, but its solution requires a tridiagonal-system solver.

For the present transient Plane Couette Flow problem, Laasonen is a suitable method when stability and robustness are more important than minimizing the computational work per time step. Since its coefficient matrix is tridiagonal, the resulting system can be solved efficiently using a specialized tridiagonal solver, which will be described separately in a separately in a later section.

F. Summary of the Laasonen scheme :

- It uses a first-order backward difference in time.
- It uses a second-order central difference in space.
- It is implicit because the unknown values at the new time level appear in the diffusion term.
- It produces a tridiagonal system at each step.
- It is first-order accurate in time and second-order accurate in space.
- It is unconditionally stable for the one-dimensional diffusion equation.
- It does not require a special initialization beyond the prescribed initial condition.
- It permits larger time steps than FTCS without violating the FTCS stability condition.

- Its linear system can be solved efficiently using a specialized tridiagonal solver.

Therefore, the Laasonen scheme provides a robust implicit alternative to the explicit FTCS, Richardson, and DuFort-Frankel schemes.

Crank-Nicolson Scheme

The Crank-Nicolson scheme is an implicit finite difference method that achieves second-order accuracy in time by evaluating the diffusion term at the arithmetic average of the current and future time levels. In contrast to the Laasonen scheme, which uses the diffusion term only at time level $n+1$, the Crank-Nicolson method blends the contributions from time levels n and $n+1$.

For the one-dimensional transient diffusion equation, the Crank-Nicolson formulation can be written as the average of the spatial diffusion term at the two adjacent time levels:

$$\left(\frac{\partial u}{\partial t}\right)_j^{n+\frac{1}{2}} = \frac{\nu}{2} \left[\left(\frac{\partial^2 u}{\partial y^2}\right)_j^{n+1} + \left(\frac{\partial^2 u}{\partial y^2}\right)_j^n \right] \quad (22)$$

This midpoint form is the key distinction between Crank-Nicolson and Laasonen. Both methods are implicit, but Crank-Nicolson is more accurate in time.

A. Time Derivative Discretization

To derive the finite difference approximation for the time derivative, expand u_j^{n+1} and u_j^n about the midpoint time level $t^{n+\frac{1}{2}}$.

Expanding u_j^{n+1} about $t^{n+\frac{1}{2}}$ gives:

$$u_j^{n+1} = u_j^{n+\frac{1}{2}} + \frac{\Delta t}{2} \left(\frac{\partial u}{\partial t}\right)_j^{n+\frac{1}{2}} + \frac{(\Delta t)^2}{8} \left(\frac{\partial^2 u}{\partial t^2}\right)_j^{n+\frac{1}{2}} + \mathcal{O}((\Delta t)^3) \quad (23)$$

Similarly, expanding u_j^n about $t^{n+\frac{1}{2}}$ gives:

$$u_j^n = u_j^{n+\frac{1}{2}} - \frac{\Delta t}{2} \left(\frac{\partial u}{\partial t}\right)_j^{n+\frac{1}{2}} + \frac{(\Delta t)^2}{8} \left(\frac{\partial^2 u}{\partial t^2}\right)_j^{n+\frac{1}{2}} - \mathcal{O}((\Delta t)^3) \quad (24)$$

Subtracting Equation (24) from Equation (23) yields:

$$u_j^{n+1} - u_j^n = \Delta t \left(\frac{\partial u}{\partial t}\right)_j^{n+\frac{1}{2}} + \mathcal{O}((\Delta t)^3) \quad (25)$$

Therefore, the midpoint time derivative can be approximated as:

$$\left(\frac{\partial u}{\partial t}\right)_j^{n+\frac{1}{2}} = \frac{u_j^{n+1} - u_j^n}{\Delta t} + \mathcal{O}((\Delta t)^2) \quad (26)$$

Thus, the Crank-Nicolson scheme is second-order accurate in time, whereas Laasonen is only first-order accurate in time.

B. Spatial Derivative Discretization

For the diffusion term, the second derivative is approximated at both time levels n and $n+1$ using the central difference formula.

At time level $n+1$:

$$\left(\frac{\partial^2 u}{\partial y^2}\right)_j^{n+1} = \frac{u_{j+1}^{n+1} - 2u_j^{n+1} + u_{j-1}^{n+1}}{(\Delta y)^2} + \mathcal{O}((\Delta y)^2) \quad (27)$$

At time level n :

$$\left(\frac{\partial^2 u}{\partial y^2}\right)_j^n = \frac{u_{j+1}^n - 2u_j^n + u_{j-1}^n}{(\Delta y)^2} + \mathcal{O}((\Delta y)^2) \quad (28)$$

Taking the arithmetic average of Equations (27) and (28) gives:

$$\frac{1}{2} \left(\frac{\partial^2 u}{\partial y^2}\right)_j^{n+1} + \frac{1}{2} \left(\frac{\partial^2 u}{\partial y^2}\right)_j^n = \frac{1}{2(\Delta y)^2} [u_{j+1}^{n+1} - 2u_j^{n+1} + u_{j-1}^{n+1} + u_{j+1}^n - 2u_j^n + u_{j-1}^n] + \mathcal{O}((\Delta y)^2) \quad (29)$$

This confirms that the Crank-Nicolson scheme retains second-order spatial accuracy, the same as Laasonen and FTCS.

C. Discrete Algebraic Equation

Substituting Equation (26) and Equation (29) into Equation (22) gives:

$$\frac{u_j^{n+1} - u_j^n}{\Delta t} = \nu \frac{1}{2(\Delta y)^2} [u_{j+1}^{n+1} - 2u_j^{n+1} + u_{j-1}^{n+1} + u_{j+1}^n - 2u_j^n + u_{j-1}^n] \quad (30)$$

Defining the diffusion number as:

$$d = \frac{\nu \Delta t}{(\Delta y)^2} \quad (31)$$

the Crank-Nicolson scheme can be rearranged into the standard implicit form:

$$-\frac{d}{2}u_{j-1}^{n+1} + (1+d)u_j^{n+1} - \frac{d}{2}u_{j+1}^{n+1} = \frac{d}{2}u_{j-1}^n + (1-d)u_j^n + \frac{d}{2}u_{j+1}^n \quad (32)$$

Equation (32) is the discrete algebraic equation for Crank-Nicolson. It has the same tridiagonal coupling pattern as Laasonen, but the right-hand side now also contains information from time level n .

D. Accuracy of the Crank-Nicolson Scheme

The temporal truncation error is:

$$\mathcal{O}((\Delta t)^2)$$

The spatial truncation error is:

$$\mathcal{O}((\Delta y)^2)$$

Therefore, the Crank-Nicolson scheme is:

- second-order accurate in time;
- second-order accurate in space.

The combined local truncation error is:

$$\mathcal{T}_{\text{CN}} = \mathcal{O}((\Delta t)^2) + \mathcal{O}((\Delta y)^2) \quad (33)$$

This is the main accuracy advantage of Crank-Nicolson over Laasonen, which is only first-order accurate in time.

E. Comparison Between Crank-Nicolson and Laasonen

The two implicit schemes used in this report are Laasonen and Crank-Nicolson. Both methods lead to tridiagonal linear systems and both are unconditionally stable for the one-dimensional diffusion equation. Their principal difference lies in how the diffusion term is treated in time.

Laasonen

- first-order accurate in time;
- second-order accurate in space;
- fully implicit at time level $n + 1$;
- more strongly damping;
- numerically robust and monotone for diffusion-dominated problems.

Crank-Nicolson

- second-order accurate in time;
- second-order accurate in space;
- averages the diffusion term between n and $n + 1$;
- less numerical damping than Laasonen;
- may show oscillations for large d .

In summary, Crank-Nicolson is more accurate than Laasonen, but Laasonen is often more diffusive and therefore more forgiving for strongly dissipative transient problems. If the goal is higher temporal accuracy, Crank-Nicolson is preferable. If the goal is stronger numerical smoothing and simpler behavior, Laasonen can be advantageous.

I. Comparison of Implicit and Explicit Schemes

The numerical schemes discussed in this report can be divided into explicit and implicit classes.

Explicit schemes

- FTCS
- Richardson
- DuFort-Frankel

Implicit schemes

- Laasonen
- Crank-Nicolson

The main differences are:

Property	Explicit schemes	Implicit schemes
Unknowns at new time level	Computed directly	Found by solving a linear system
Stability	Usually restricted or problematic	Typically unconditionally stable for diffusion
Computational cost per step	Low	Higher
Time-step flexibility	Limited by stability	Much larger
Implementation	Simpler	More involved
Accuracy control	Often lower or unstable	Can be higher and more robust

For the present diffusion problem:

- FTCS is simple but conditionally stable.
- Richardson is formally attractive but unusable because it is unconditionally unstable.

- DuFort-Frankel is explicit and unconditionally stable, but it uses three time levels and requires a startup procedure.
- Laasonen is robust and unconditionally stable, but only first-order accurate in time.
- Crank-Nicolson is the most accurate among the schemes considered here, with second-order accuracy in both time and space, but it may require care when large time steps create oscillatory behavior.

Therefore, the implicit schemes provide better stability properties, while the explicit schemes provide simpler updates but weaker stability guarantees.

J. Summary of the Crank-Nicolson Scheme

The main properties of the Crank-Nicolson scheme are:

- It is an implicit finite difference method.
- It evaluates the diffusion term at the average of time levels n and $n + 1$.
- It is second-order accurate in time.
- It is second-order accurate in space.
- It produces a tridiagonal linear system at every time step.
- It is unconditionally stable for the one-dimensional diffusion equation.
- It may produce oscillations for large diffusion numbers.
- It is more accurate in time than Laasonen.
- It requires a tridiagonal solver such as the Thomas algorithm.

Therefore, Crank-Nicolson provides a high-accuracy implicit alternative for the transient Plane Couette Flow problem, especially when accuracy is more important than the extra damping provided by Laasonen.

Linear System Formulation and Dirichlet Boundary Conditions

In implicit finite difference formulations for the one-dimensional transient diffusion equation, the spatial coupling of unknown nodal values at the advanced time level $n + 1$ leads to a coupled system of linear algebraic equations. This section presents the general coefficient matrix assembled for the interior nodes and establishes how Dirichlet boundary conditions are integrated into the algebraic framework.

A. System of Equations for Interior Nodes

Consider a one-dimensional domain discretized into N grid points indexed by $j = 0, 1, \dots, N - 1$. The physical boundaries are located at $j = 0$ (lower wall) and $j = N - 1$ (upper wall). Consequently, the interior computational domain consists of $N - 2$ unknown nodes:

$$j = 1, 2, \dots, N - 2$$

Evaluating an implicit discretization stencil at each interior node couples the unknown velocity u_j^{n+1} to its adjacent spatial neighbors u_{j-1}^{n+1} and u_{j+1}^{n+1} . For instance, employing the standard fully implicit (Laasonen) scheme yields the following discrete equation at node j :

$$-d u_{j-1}^{n+1} + (1 + 2d) u_j^{n+1} - d u_{j+1}^{n+1} = u_j^n \quad (34)$$

where the dimensionless diffusion number d is defined as:

$$d = \frac{\nu \Delta t}{(\Delta y)^2} \quad (35)$$

Writing Equation (34) for all interior nodes $j = 1, 2, \dots, N - 2$ simultaneously produces an $(N - 2) \times (N - 2)$ linear system of equations:

$$\begin{bmatrix} 1 + 2d & -d & 0 & \cdots & 0 \\ -d & 1 + 2d & -d & \ddots & \vdots \\ 0 & -d & 1 + 2d & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & -d \\ 0 & \cdots & 0 & -d & 1 + 2d \end{bmatrix} \begin{bmatrix} u_1^{n+1} \\ u_2^{n+1} \\ u_3^{n+1} \\ \vdots \\ u_{N-2}^{n+1} \end{bmatrix} = \begin{bmatrix} u_1^n \\ u_2^n \\ u_3^n \\ \vdots \\ u_{N-2}^n \end{bmatrix} + \begin{bmatrix} d u_0^{n+1} \\ 0 \\ \vdots \\ 0 \\ d u_{N-1}^{n+1} \end{bmatrix} \quad (36)$$

In Equation (36), the boundary contributions u_0^{n+1} and u_{N-1}^{n+1} appear directly on the right-hand side because their values are prescribed by the boundary conditions.

B. Incorporation of Dirichlet Boundary Conditions

For the physical Couette flow problem, the boundary values at the lower and upper boundaries are strictly prescribed by constant Dirichlet boundary conditions:

$$u_0^{n+1} = U_{\text{bottom}}, \quad u_{N-1}^{n+1} = U_{\text{top}} \quad (37)$$

There are two primary approaches for incorporating these boundary conditions into the coefficient matrix and algebraic system.

Approach A: Reduced Interior System $((N - 2) \times (N - 2))$

In the reduced interior formulation, the known boundary values are excluded from the unknown solution vector. As shown in Equation (36), their contributions are absorbed directly into the right-hand side vector:

$$\mathbf{A}_{\text{int}} \mathbf{u}_{\text{int}}^{n+1} = \mathbf{b}_{\text{int}}^n \quad (38)$$

where $\mathbf{u}_{\text{int}}^{n+1} = [u_1^{n+1}, u_2^{n+1}, \dots, u_{N-2}^{n+1}]^T$ is of dimension $N - 2$, and the right-hand side vector elements are given by:

$$\begin{aligned}
b_1 &= u_1^n + d U_{\text{bottom}} \\
b_j &= u_j^n \quad \text{for } j = 2, 3, \dots, N-3 \\
b_{N-2} &= u_{N-2}^n + d U_{\text{top}}
\end{aligned} \tag{39}$$

This approach minimizes the algebraic system to $(N-2)$ equations, but it introduces an index offset between the algebraic vector index (from 0 to $N-3$) and the physical spatial grid index (from 1 to $N-2$).

Approach B: Global Formulation ($N \times N$)

In the global system formulation, all computational nodes across the domain—including the boundary nodes—are preserved in the solution vector $\mathbf{u}^{n+1} \in \mathbb{R}^N$.

To close the system, the first and last rows of the coefficient matrix are formulated as identity equations corresponding to the Dirichlet boundary conditions in Equation (37):

$$\begin{aligned}
\text{Row } 0 : \quad & 1 \cdot u_0^{n+1} + 0 \cdot u_1^{n+1} + \dots + 0 \cdot u_{N-1}^{n+1} = U_{\text{bottom}} \\
\text{Row } N-1 : \quad & 0 \cdot u_0^{n+1} + \dots + 0 \cdot u_{N-2}^{n+1} + 1 \cdot u_{N-1}^{n+1} = U_{\text{top}}
\end{aligned} \tag{40}$$

Assembling the complete $N \times N$ global system yields:

$$\begin{bmatrix}
1 & 0 & 0 & \dots & 0 & 0 \\
-d & 1+2d & -d & \dots & 0 & 0 \\
0 & -d & 1+2d & \ddots & \vdots & \vdots \\
\vdots & \ddots & \ddots & \ddots & -d & 0 \\
0 & \dots & 0 & -d & 1+2d & -d \\
0 & 0 & \dots & 0 & 0 & 1
\end{bmatrix}
\begin{bmatrix}
u_0^{n+1} \\
u_1^{n+1} \\
u_2^{n+1} \\
\vdots \\
u_{N-2}^{n+1} \\
u_{N-1}^{n+1}
\end{bmatrix}
=
\begin{bmatrix}
U_{\text{bottom}} \\
u_1^n \\
u_2^n \\
\vdots \\
u_{N-2}^n \\
U_{\text{top}}
\end{bmatrix} \tag{41}$$

C. Selection for C++ Implementation

While Equation (38) and Equation (41) are mathematically equivalent for all interior nodes, the global $N \times N$ formulation defined in Equation (41) is adopted in the current C++ solver architecture.

The primary advantages of this formulation are:

1. **Uniform Indexing:** The row index of the algebraic system corresponds directly to the physical grid index j , which eliminates off-by-one indexing errors.
2. **Simplified Memory Layout:** The solution vector contains the complete domain profile, eliminating the need to manually reinsert boundary values after each linear solve.
3. **Boundary Condition Extensibility:** Alternative boundary conditions (such as Neumann or Robin boundaries) can be implemented simply by adjusting the discrete coef-

ficients in row 0 or row $N - 1$, without altering the dimension or structural mapping of the linear solver.

Linear Algebraic Solvers

Mathematical Formulation of the Linear System

In the context of implicit finite difference discretizations for transient partial differential equations (such as the 1D diffusion equation), the advancement from time level n to $n + 1$ requires solving a coupled system of algebraic equations across all interior grid points simultaneously. In general algebraic form, this linear system is expressed as:

$$\mathbf{A}\mathbf{x}^{n+1} = \mathbf{b}^n \quad (42)$$

where:

- $\mathbf{A} \in \mathbb{R}^{N \times N}$ is the square coefficient matrix of size $N \times N$,
- $\mathbf{x}^{n+1} \in \mathbb{R}^N$ is the vector of unknown field variables at time level $n + 1$ (i.e., $\mathbf{u}^{n+1}$),
- $\mathbf{b}^n \in \mathbb{R}^N$ is the right-hand side (RHS) load vector containing the known solution from time level n along with prescribed boundary conditions.

In expanded matrix-vector notation, Equation (42) is represented as:

$$\begin{bmatrix} a_{0,0} & a_{0,1} & a_{0,2} & \cdots & a_{0,N-1} \\ a_{1,0} & a_{1,1} & a_{1,2} & \cdots & a_{1,N-1} \\ a_{2,0} & a_{2,1} & a_{2,2} & \cdots & a_{2,N-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{N-1,0} & a_{N-1,1} & a_{N-1,2} & \cdots & a_{N-1,N-1} \end{bmatrix} \begin{bmatrix} x_0^{n+1} \\ x_1^{n+1} \\ x_2^{n+1} \\ \vdots \\ x_{N-1}^{n+1} \end{bmatrix} = \begin{bmatrix} b_0^n \\ b_1^n \\ b_2^n \\ \vdots \\ b_{N-1}^n \end{bmatrix} \quad (43)$$

Solving Equation (42) accurately and efficiently is central to the execution of any implicit scheme.

From a general linear algebra perspective, treating the coefficient matrix $\mathbf{A}$ in its full algebraic representation necessitates the deployment of general direct solvers, such as **Gaussian Elimination**, which will be discussed in detail in subsequent sections. However, applying standard Gaussian Elimination to a dense $N \times N$ system incurs an asymptotic computational complexity of $\mathcal{O}(N^3)$ operations and requires $\mathcal{O}(N^2)$ storage.

Upon closer inspection of the governing discrete algebraic equations, the tridiagonal structure of the coefficient matrix $\mathbf{A}$ is mathematically inevitable. The spatial second-derivative operator is approximated using a compact three-point central difference stencil, which inherently couples the state variable at node j exclusively to its immediate spatial neighbors, $j - 1$ and $j + 1$. Because no long-range node interactions exist in the local differential stencil, the off-diagonal entries vanish identically beyond the first sub- and super-diagonals.

Consequently, the resulting system can be solved straightforwardly and efficiently using the **Thomas Algorithm** (Tridiagonal Matrix Algorithm, TDMA)—a streamlined variant of Gaussian Elimination that eliminates unnecessary operations on zero entries. This algorithm resolves the system in two simple passes (a forward elimination sweep followed by a backward substitution sweep), executing in $\mathcal{O}(N)$ computational time with $\mathcal{O}(N)$ memory storage. Furthermore, because the coefficient matrix generated by this implicit scheme is strictly diagonally dominant under standard conditions, the Thomas Algorithm is guaranteed to be unconditionally stable without requiring numerical pivoting.

Gaussian Elimination with Partial Pivoting

Standard Gaussian Elimination Overview

Standard Gaussian Elimination is a classical direct method for solving a linear system of algebraic equations $\mathbf{Ax} = \mathbf{b}$. The core idea is to transform the system into an equivalent, simpler form without changing its solution. This is accomplished in two sequential stages:

1. **Forward Elimination:** Using elementary row operations, the augmented matrix $[\mathbf{A} \mid \mathbf{b}]$ is converted into an upper-triangular form $[\mathbf{U} \mid \mathbf{c}]$ by eliminating all coefficients located below the main diagonal.
2. **Back Substitution:** Starting from the last row (which contains only one unknown), the solution vector $\mathbf{x}$ is calculated backwards, step-by-step, up to the first variable.

What is the Pivot Element?

During Forward Elimination at stage k (working on the k -th column), the diagonal entry $a_{k,k}$ is called the **pivot element** (or simply the **pivot**), and the corresponding row k is referred to as the **pivot row**.

To eliminate an entry $a_{i,k}$ located below the pivot in row i ($i > k$), we calculate an elimination factor known as the **multiplier**:

$$m_{i,k} = \frac{a_{i,k}}{a_{k,k}}$$

We then subtract $m_{i,k}$ times the pivot row from row i to introduce a zero at position (i, k) .

Why Do We Need Partial Pivoting?

In the standard approach, we directly divide by the diagonal element $a_{k,k}$. This creates two major computational problems:

1. **Division by Zero:** If $a_{k,k} = 0$, the multiplier $m_{i,k}$ cannot be computed, causing the algorithm to crash completely—even if the system has a unique, well-defined solution.
2. **Round-off Error Amplification:** If $a_{k,k}$ is not zero but is very small in magnitude compared to the coefficients below it ($|a_{k,k}| \ll |a_{i,k}|$), the multiplier becomes very large ($|m_{i,k}| \gg 1$). In finite-precision floating-point arithmetic, multiplying an entire row by a large multiplier magnifies round-off errors and leads to severe numerical inaccuracies.

The Partial Pivoting Strategy

To resolve these issues, **Partial Pivoting** (row pivoting) is applied at each elimination step k :

1. **Search for the Maximum Magnitude:** Before eliminating column k , the algorithm scans all elements in column k from the diagonal down to the bottom row to locate the coefficient with the largest absolute value:

$$p = \arg \max_{m \in \{k, k+1, \dots, N-1\}} |a_{m,k}|$$

2. **Row Swap:** If this maximum element is located in a row p different from row k ($p \neq k$), row k and row p are completely swapped across both matrix $\mathbf{A}$ and vector $\mathbf{b}$.

By placing the largest available entry on the diagonal, every multiplier is guaranteed to satisfy:

$$|m_{i,k}| = \left| \frac{a_{i,k}}{a_{k,k}} \right| \leq 1, \quad \forall i > k$$

This bounds the multipliers, prevents division by zero, and ensures numerical stability throughout the computation.

C++ Implementation of Gaussian Elimination with Partial Pivoting

This section describes the object-oriented implementation of the Gaussian Elimination algorithm with Partial Pivoting in the `GaussianElimination` class.

The implementation follows the **Separation of Concerns** principle. Each important step of the algorithm is encapsulated in a separate method, making the code easier to understand, test, maintain, and extend.

FindPivotRowIndex Method

Conceptual Role Of FindPivotRowIndex The objective of this method is to implement the row pivot selection strategy during elimination step k . To minimize round-off errors and prevent potential division by zero, the method traverses column k from the main diagonal entry (k, k) down to the final row $(N-1, k)$, identifying the row that contains the maximum absolute coefficient:

$$\text{PivotRow} = \arg \max_{i \in \{k, k+1, \dots, N-1\}} |A_{i,k}|$$

C++ Implementation Of FindPivotRowIndex

```
sstd::size_t GaussianElimination::FindPivotRowIndex(  
    const CoefficientMatrix& A,  
    const std::size_t PivotIndex)
```

```

{
    const std::size_t NumRows = A.GetNumRows();

    // Ensure the pivot index is within matrix bounds
    if (PivotIndex >= NumRows)
    {
        throw std::out_of_range("FindPivotRowIndex: PivotIndex is out of range.");
    }

    // Initialize the best candidate with the current diagonal element
    std::size_t CandidateRowIndex = PivotIndex;
    double MaxCandidateAbsValue = std::fabs(A.GetValue(PivotIndex, PivotIndex));

    // Search subsequent rows for the largest absolute pivot value
    for (std::size_t RowIndex = PivotIndex + 1; RowIndex < NumRows; ++RowIndex)
    {
        const double CurrentAbsValue = std::fabs(A.GetValue(RowIndex, PivotIndex));

        if (CurrentAbsValue > MaxCandidateAbsValue)
        {
            MaxCandidateAbsValue = CurrentAbsValue;
            CandidateRowIndex = RowIndex;
        }
    }

    return CandidateRowIndex;
}

```

FindPivotRow Method Summary The FindPivotRowIndex method searches the active column for the largest absolute coefficient, starting from the current pivot row. It returns the index of the selected row for use in partial pivoting without modifying the matrix.

SwapRows Method

Conceptual Role Of SwapRows The SwapRows method exchanges two rows of the coefficient matrix. During Gaussian Elimination with Partial Pivoting, it is used to move the selected pivot row into the current pivot position.

This operation ensures that the row containing the largest suitable pivot coefficient is placed at the correct position before the elimination process continues.

C++ Implementation Of SwapRows Method

```

void GaussianElimination::SwapRows(
    CoefficientMatrix& A,
    RHS& RHS,

```

```

    std::size_t FirstRowIndex,
    std::size_t SecondRowIndex)
{
    const std::size_t NumCols = A.GetNumColumns();

    if (FirstRowIndex >= A.GetNumRows() || SecondRowIndex >= A.GetNumRows())
    {
        throw std::out_of_range("SwapRows: row index is out of range.");
    }

    if (RHS.GetSize() != A.GetNumRows())
    {
        throw std::invalid_argument("SwapRows: size of RHS must match matrix size.");
    }

    if (FirstRowIndex == SecondRowIndex)
    {
        return;
    }

    for (std::size_t ColIndex = 0; ColIndex < NumCols; ++ColIndex)
    {
        const double Temp = A.GetValue(FirstRowIndex, ColIndex);
        A.SetValue(FirstRowIndex, ColIndex, A.GetValue(SecondRowIndex, ColIndex));
        A.SetValue(SecondRowIndex, ColIndex, Temp);
    }

    const double TempRHS = RHS.GetValue(FirstRowIndex);
    RHS.SetValue(FirstRowIndex, RHS.GetValue(SecondRowIndex));
    RHS.SetValue(SecondRowIndex, TempRHS);
}

```

SwapRows Method Summary The `SwapRows` method does not perform any complex calculation. It simply exchanges the coefficients of two rows, which is equivalent to swapping the positions of two equations in the linear system. In this method, `FirstRowIndex` represents the current pivot row in the elimination process, while `SecondRowIndex` corresponds to the row identified by `FindPivotRowIndex` that contains the largest absolute coefficient in the active column. The method simply swaps these two rows so that the optimal pivot is placed on the main diagonal.

ForwardElimination Method

Conceptual Role Of ForwardElimination The objective of the `ForwardElimination` method is to transform the augmented matrix $[A|b]$ into an equivalent upper triangular matrix $[U|c]$ through systematic row operations.

During each step k , the method incorporates **Partial Pivoting** by delegating pivot row selection to `FindPivotRowIndex` and row interchange to `SwapRows`. Once the optimal pivot element $A_{k,k}$ is placed on the main diagonal, multipliers (factors) are computed to eliminate all sub-diagonal entries in the active column k :

$$m_{i,k} = \frac{A_{i,k}}{A_{k,k}}, \quad \text{for } i = k + 1, \dots, N - 1$$

The row reduction updates both the coefficient matrix and the right-hand side vector:

$$A_{i,j} \leftarrow A_{i,j} - m_{i,k}A_{k,j}, \quad \text{for } j = k + 1, \dots, N - 1$$

$$b_i \leftarrow b_i - m_{i,k}b_k$$

C++ Implementation Of ForwardElimination Method

```
void GaussianElimination::ForwardElimination(
    CoefficientMatrix& A,
    RHS& RHS)
{
    const std::size_t NumRows = A.GetNumRows();
    const std::size_t NumCols = A.GetNumColumns();

    if (NumRows != NumCols)
    {
        throw std::invalid_argument("ForwardElimination: matrix A must be square.");
    }

    if (RHS.GetSize() != NumRows)
    {
        throw std::invalid_argument("ForwardElimination: size of RHS must match matrix s");
    }

    if (NumRows == 0)
    {
        return;
    }

    for (std::size_t PivotColIndex = 0; PivotColIndex < NumCols - 1; ++PivotColIndex)
    {
        std::size_t PivotRowIndex = PivotColIndex;

        const std::size_t Candidate_PivotRowIndex = FindPivotRowIndex(A, PivotColIndex);
```

```

    if (Candidate_PivotRowIndex != PivotRowIndex)
    {
        SwapRows(A, RHS, PivotRowIndex, Candidate_PivotRowIndex);
    }

    const double PivotValue = A.GetValue(PivotRowIndex, PivotColIndex);

    if (std::fabs(PivotValue) < 1.0e-14)
    {
        throw std::runtime_error("ForwardElimination: singular or nearly singular ma

    for (std::size_t RowIndex = PivotRowIndex + 1; RowIndex < NumRows; ++RowIndex)
    {
        const double Factor = A.GetValue(RowIndex, PivotColIndex) / PivotValue;

        A.SetValue(RowIndex, PivotColIndex, 0.0);

        for (std::size_t ColIndex = PivotColIndex + 1; ColIndex < NumCols; ++ColIndex)
        {
            const double NewValue =
                A.GetValue(RowIndex, ColIndex) - Factor * A.GetValue(PivotRowIndex,
                A.SetValue(RowIndex, ColIndex, NewValue);
        }

        const double NewRHS =
            RHS.GetValue(RowIndex) - Factor * RHS.GetValue(PivotRowIndex);
        RHS.SetValue(RowIndex, NewRHS);
    }
}

if (std::fabs(A.GetValue(NumRows - 1, NumRows - 1)) < 1.0e-14)
{
    throw std::runtime_error("ForwardElimination: zero pivot found on last row.");
}
}

```

ForwardElimination Method Summary The ForwardElimination method transforms the system into upper-triangular form using partial pivoting. It systematically computes row elimination multipliers, zeroes out sub-diagonal coefficients, and simultaneously updates the right-hand side vector RHS while detecting singular matrices.

ForwardElimination — Transforming the System The ForwardElimination method transforms the general dense linear system of Equation(43) into an equivalent upper triangular form shown in Equation (45).

Upper Triangular System Following the completion of the forward elimination sweeps, all entries strictly below the main diagonal vanish ($a'_{i,j} = 0$ for $i > j$):

$$\begin{bmatrix} a'_{0,0} & a'_{0,1} & a'_{0,2} & \cdots & a'_{0,N-1} \\ 0 & a'_{1,1} & a'_{1,2} & \cdots & a'_{1,N-1} \\ 0 & 0 & a'_{2,2} & \cdots & a'_{2,N-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & a'_{N-1,N-1} \end{bmatrix} \begin{bmatrix} x_0^{n+1} \\ x_1^{n+1} \\ x_2^{n+1} \\ \vdots \\ x_{N-1}^{n+1} \end{bmatrix} = \begin{bmatrix} b_0'^n \\ b_1'^n \\ b_2'^n \\ \vdots \\ b_{N-1}'^n \end{bmatrix} \quad (44)$$

BackSubstitution Method

Conceptual Role Of BackSubstitution Once the forward elimination process transforms the original coefficient matrix $\mathbf{A}$ into an upper triangular matrix $\mathbf{U}$ and updates the right-hand side vector $\mathbf{b}$ accordingly, the linear system takes the form:

$$\mathbf{U}\mathbf{x} = \mathbf{b} \quad (45)$$

Expanded into individual scalar equations, the i -th row equation is written as:

$$U_{i,i} x_i + \sum_{j=i+1}^{N-1} U_{i,j} x_j = b_i \quad (46)$$

The back-substitution procedure resolves the unknowns in reverse topological order, starting from the last unknown x_{N-1} and progressing backwards to x_0 . Isolating x_i yields the explicit mathematical recurrence relation:

$$x_i = \frac{b_i - \sum_{j=i+1}^{N-1} U_{i,j} x_j}{U_{i,i}}, \quad \text{for } i = N-1, N-2, \dots, 0 \quad (47)$$

Where:

- i : The current row and equation index, traversed backwards from $N-1$ down to 0.
- j : Column indices strictly to the right of the main diagonal ($j > i$).
- $U_{i,i}$: The non-zero pivot entry on the main diagonal of the upper triangular system.
- c_i : The corresponding updated load value in the right-hand side vector.
- $\sum_{j=i+1}^{N-1} U_{i,j} x_j$: The cumulative contribution of previously computed unknowns in subsequent rows.
- x_i : The newly determined scalar solution for the i -th degree of freedom.

C++ Implementation Of BackSubstitution The implementation employs a signed index type (`std::ptrdiff_t`) to eliminate the risk of unsigned underflow during reverse iteration. Furthermore, singular or near-singular systems are trapped by verifying that $|U_{i,i}| \geq 1.0 \times 10^{-14}$ before performing the floating-point division.

```
// Mathematical formulation of back-substitution:
//  $x_i = (b_i - \sum_{j=i+1}^{N-1} A_{i,j} * x_j) / A_{i,i}$ 
//
// Where:
//  $i$  : Current row and equation index (from  $N-1$  down to  $0$ )
//  $j$  : Column indices to the right of the main diagonal ( $j > i$ )
//  $A_{i,i}$  : Pivot/diagonal entry of the current row
//  $b_i$  : Corresponding element in the right-hand side (RHS) vector
//  $sum\_upper$  : Cumulative sum of known upper terms:  $\sum_{j=i+1}^{N-1} A_{i,j} * x_j$ 
//  $x_i$  : Computed unknown value stored in the solution vector

void GaussianElimination::BackSubstitution(
    const CoefficientMatrix& A,
    const RHS& RHS,
    Field1D& Solution)
{
    const std::size_t NumRows = A.GetNumRows();
    const std::size_t NumCols = A.GetNumColumns();

    if (NumRows != NumCols)
    {
        throw std::invalid_argument("BackSubstitution: matrix A must be square.");
    }

    if (RHS.GetSize() != NumRows || Solution.Size() != NumRows)
    {
        throw std::invalid_argument("BackSubstitution: size of RHS and Solution must match");
    }

    // Mathematical formula:
    //  $x_i = (b_i - \sum_{j=i+1}^{N-1} A_{i,j} * x_j) / A_{i,i}$ 

    // Iterate backward from the last row ( $N-1$ ) down to the first row ( $0$ )
    for (std::ptrdiff_t row = static_cast<std::ptrdiff_t>(NumRows) - 1; row >= 0; --row)
    {
        const std::size_t i = static_cast<std::size_t>(row);

        // Calculate the sum of known upper terms:  $\sum_{j=i+1}^{N-1} A_{i,j} * x_j$ 
        double sum_upper = 0.0;
        for (std::size_t j = i + 1; j < NumCols; ++j)
```

```

    {
        sum_upper += A.GetValue(i, j) * Solution.GetValue(j);
    }

    const double A_ii = A.GetValue(i, i);

    if (std::fabs(A_ii) < 1.0e-14)
    {
        throw std::runtime_error("BackSubstitution: zero diagonal entry detected.");
    }

    const double b_i = RHS.GetValue(i);

    // x_i = (b_i - sum_upper) / A_ii
    Solution.SetValue(i, (b_i - sum_upper) / A_ii);
}
}

```

Thomas Algorithm

The **Thomas Algorithm**, also known as the **Tridiagonal Matrix Algorithm (TDMA)**, is a specialized direct solver for linear systems of the form $A\mathbf{x} = \mathbf{b}$, where A is a tridiagonal matrix. It is essentially a stripped-down version of Gaussian Elimination that exploits the banded structure of the coefficient matrix, reducing both memory and arithmetic cost dramatically.

In general Gaussian Elimination, the algorithm operates on a full $N \times N$ matrix, requiring $\mathcal{O}(N^3)$ operations and $\mathcal{O}(N^2)$ storage. For a tridiagonal system, however, only three diagonals carry non-zero entries. The Thomas Algorithm discards the full-matrix machinery entirely and works exclusively with three one-dimensional vectors:

- **lower** — the sub-diagonal (lower diagonal) coefficients,
- **main** — the main diagonal coefficients,
- **upper** — the super-diagonal (upper diagonal) coefficients.

This representation reduces storage from $\mathcal{O}(N^2)$ to $\mathcal{O}(N)$ and operation count from $\mathcal{O}(N^3)$ to $\mathcal{O}(N)$, making it the standard choice for the implicit finite-difference schemes that produce tridiagonal systems at every time step — such as the Laasonen and Crank-Nicolson methods applied throughout this project.

The algorithm proceeds in two passes: a **Forward Elimination** phase that sweeps downward to eliminate the sub-diagonal entries, and a **Back Substitution** phase that recovers the solution vector from the resulting upper-bidiagonal system.

Forward Elimination: Matrix Transformation

The general tridiagonal system $Ax = b$ of size N takes the following form before elimination:

$$\begin{bmatrix} d_0 & u_0 & & & \\ \ell_1 & d_1 & u_1 & & \\ & \ell_2 & d_2 & \ddots & \\ & & \ddots & \ddots & u_{N-2} \\ & & & \ell_{N-1} & d_{N-1} \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ \vdots \\ x_{N-1} \end{bmatrix} = \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ \vdots \\ b_{N-1} \end{bmatrix} \quad (48)$$

At each step $i = 1, 2, \dots, N - 1$, the elimination factor is:

$$f_i = \frac{\ell_i}{d_{i-1}} \quad (49)$$

and the main diagonal and right-hand side are updated as:

$$d_i^* = d_i - f_i \cdot u_{i-1}, \quad b_i^* = b_i - f_i \cdot b_{i-1} \quad (50)$$

After Forward Elimination completes, the lower diagonal is eliminated and the system reduces to an upper bidiagonal form:

$$\begin{bmatrix} d_0 & u_0 & & & \\ & d_1^* & u_1 & & \\ & & d_2^* & \ddots & \\ & & & \ddots & u_{N-2} \\ & & & & d_{N-1}^* \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ \vdots \\ x_{N-1} \end{bmatrix} = \begin{bmatrix} b_0 \\ b_1^* \\ b_2^* \\ \vdots \\ b_{N-1}^* \end{bmatrix} \quad (51)$$

Note that d_0 and b_0 serve as the anchor of the sweep and remain unchanged, while u_i is unmodified throughout. The resulting upper bidiagonal system is immediately ready for Back Substitution.

C++ Implementation Of Forward Elimination

The `ForwardElimination` function implements the forward sweep of the **Thomas Algorithm**. Starting from row $i = 1$ and iterating to $i = N - 1$, it computes the elimination factor $f_i = \ell_i/d_{i-1}$, updates the diagonal as $d_i \leftarrow d_i - f_i \cdot u_{i-1}$, modifies the RHS as $b_i \leftarrow b_i - f_i \cdot b_{i-1}$, and zeros out the sub-diagonal entry ℓ_i . The result is an upper bidiagonal system ready for back substitution.

```
void ThomasAlgorithm::ForwardElimination(
    TridiagonalMatrix& TridiagonalMatrix , RHS& RHS_Obj)
{
    const std::size_t NumRows = TridiagonalMatrix.SizeM();

    if (NumRows == 0)
    {
        throw std::invalid_argument(
```

```

        "ForwardElimination: diagonal vectors must not be empty.");
    }

    if (TridiagonalMatrix.SizeL() != NumRows
        || TridiagonalMatrix.SizeU() != NumRows
        || RHS_Obj.GetSize() != NumRows)
    {
        throw std::invalid_argument(
            "ForwardElimination: all diagonal vectors and RHS must have the same size.");
    }

    for (std::size_t RowIndex = 1; RowIndex < NumRows; ++RowIndex)
    {
        const double OriginalDiagonalValue = TridiagonalMatrix.GetMidValue(RowIndex - 1);

        if (std::fabs(OriginalDiagonalValue) < 1.0e-14)
        {
            throw std::runtime_error(
                "ForwardElimination: zero or nearly zero diagonal entry detected.");
        }

        const double Factor = TridiagonalMatrix.GetLValue(RowIndex) / OriginalDiagonalValue;

        const double NewDiagonal = TridiagonalMatrix.GetMidValue(RowIndex) - Factor * TridiagonalMatrix.GetLValue(RowIndex - 1);
        TridiagonalMatrix.SetMValue(RowIndex, NewDiagonal);

        const double NewRHS = RHS_Obj.GetValue(RowIndex) - Factor * RHS_Obj.GetValue(RowIndex - 1);
        RHS_Obj.SetValue(RowIndex, NewRHS);

        TridiagonalMatrix.SetLValue(RowIndex, 0.0);
    }
}

```

C++ Implementation Of Back Substitution

Once forward elimination reduces the system to upper-bidiagonal form, back substitution recovers the solution vector. The last unknown is solved directly:

$$x_{N-1} = \frac{b_{N-1}^*}{d_{N-1}^*}$$

Then, for $i = N - 2$ down to 0:

$$x_i = \frac{b_i^* - u_i \cdot x_{i+1}}{d_i^*}$$

The loop variable uses `std::ptrdiff_t` instead of `std::size_t` to allow an explicit ≥ 0 termination condition and avoid integer underflow on the unsigned decrement. A zero-pivot check is performed at each step before the division.

```
void ThomasAlgorithm::BackSubstitution(
    const TridiagonalMatrix& TridiagonalMatrix,
    const RHS& RHS_Obj,
    Field1D& Solution)
{
    const std::size_t NumRows = TridiagonalMatrix.SizeM();

    if (NumRows == 0)
    {
        throw std::invalid_argument(
            "ThomasAlgorithm::BackSubstitution: "
            "diagonal vectors must not be empty.");
    }

    if (TridiagonalMatrix.SizeU() != NumRows
        || RHS_Obj.GetSize() != NumRows
        || Solution.Size() != NumRows)
    {
        throw std::invalid_argument(
            "ThomasAlgorithm::BackSubstitution: "
            "all vectors must have the same size.");
    }

    const double LastDiagonalValue = TridiagonalMatrix.GetMidValue(NumRows - 1);

    if (std::fabs(LastDiagonalValue) < 1.0e-14)
    {
        throw std::runtime_error(
            "ThomasAlgorithm::BackSubstitution: "
            "zero or near-zero pivot encountered.");
    }

    Solution.SetValue(NumRows - 1, RHS_Obj.GetValue(NumRows - 1) / LastDiagonalValue);

    for (std::ptrdiff_t RowIndex = static_cast<std::ptrdiff_t>(NumRows) - 2; RowIndex >= 0; --RowIndex)
    {
```

```

    const std::size_t i = static_cast<std::size_t>(RowIndex);

    const double DiagonalValue = TridiagonalMatrix.GetMidValue(i);

    if (std::fabs(DiagonalValue) < 1.0e-14)
    {
        throw std::runtime_error(
            "ThomasAlgorithm::BackSubstitution: "
            "zero or near-zero pivot encountered.");
    }

    const double NewSolution =(RHS_Obj.GetValue(i) - TridiagonalMatrix.GetUValue(i)*
        / DiagonalValue;

    Solution.SetValue(i, NewSolution);
}
}

```